

APUNTES DEL 08/08/26

José Emilio Alvarado Méndez

08 / 08 / 26 · PROF. DR. FRANCISCO J. TORRES-ROJAS

DONDE ESTAMOS?!

```
int x = 42;
```

SCANNER

¡estamos aquí! :]

INT

ID

ASSIGN

INTLITERAL

SEMICOLON

PARSER

CONTENIDO DE LOS APUNTES

01
02
03

01

Estructuras de datos del compilador

¿Que información guarda un compilador mientras compila? (token, arbol, tablas, código intermedio, archivos temporales.)

PÁG. 03 - 15

02

Análisis léxico

Todo lo que involucra pasar del caracter al token. (scanner, lexemas, interacción con el parser, espacios, Ejemplos de FORTRAN, lookahead y maximal munch)

PÁG. 16 - 35

03

Hacia lenguajes formales

De la detección de tokens a la descripción formal de patrones.. más que todo un “segway” hacia la siguiente clase. :)

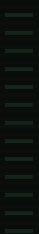
PÁG. 36 - 39 · CIERRE 40 - 42

—
—
—

NIVELES DE CONTENIDO

Estos apuntes mezclan lo que se dijo en clase con cosas que agregué después. Lo marqué con barras al inicio del bloque para lograr distinguirlo.

01



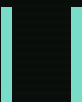
02



03



Contenido dicho en clase palabra por palabra o parafraseado, casi todo lo que sigue es de este nivel, así que solo aparece marcado donde conviene distinguirlo.



Aclaraciones o contexto que agregué para que lo anterior se entienda mejor.



Ampliación, ejemplo opcional o información complementaria.

01 ESTRUCTURAS DE DATOS DEL COMPILADOR

Qué información mantiene un compilador mientras traduce, y por qué se toman estas decisiones de implementación.

01 • ESTRUCTURAS DE DATOS

PÁG. 03 - 15

02 • ANÁLISIS LÉXICO

PÁG. 16 - 35

03 • HACIA LENGUAJES FORMALES

PÁG. 36 - 39



DE LAS ESTRUCTURAS INTERNAS DEL COMPILADOR AL ANÁLISIS LÉXICO

PREGUNTA 1

¿Qué información mantiene un compilador mientras trabaja?

Tokens, árboles sintácticos, tabla de símbolos, tabla de literales, código intermedio y archivos auxiliares. Son las estructuras donde el compilador va dejando lo que “entendió”.

PREGUNTA 2

¿Cómo empieza a entender un programa?

Leyendo caracteres y agrupándolos en unidades léxicas significativas. Ese es el trabajo del scanner, y es uno de los ejes centrales de esta clase.

FUENTE

caracteres

->

ANÁLISIS LÉXICO

aquí tamos :]

->

TOKENS

categorías

->

PARSER

estructura

->

SEMÁNTICA / R.I.

significado

LÉXICO, SINTAXIS Y SEMÁNTICA

• LÉXICO

¿Qué unidades existen?

Reconoce las piezas mínimas con significado como identificadores, números, operadores, delimitadores o palabras reservadas.

• SINTAXIS

¿Cómo pueden combinarse?

Define que secuencias de tokens forman construcciones válidas del lenguaje. Pueden ser una asignación, una condición, un ciclo, etc.

• SEMÁNTICA

¿Qué significan y son coherentes?

Revisa si tiene sentido, osea si la variable existe, si los tipos concuerdan, si la operación tiene una interpretación válida, etc.

ENTRADA

```
total = x + 25;
```

TOKENS APROXIMADOS

ID

ASSIGN

ID

PLUS

INTLITERAL

SEMICOLON

Léxico = las piezas. Sintaxis = cómo encajan. Semántica = qué significan. (básicamente) :]

ESTRUCTURAS DE DATOS DE UN COMPILADOR

Un compilador no traduce de un solo p~~igolpe~~zo, sino que va guardando resultados parciales. Estas son las estructuras típicas que aparecen en el camino.

01

TOKEN

Unidad léxica ya clasificada. Lo que el scanner, con mucho amor y cariño, entrega al parser.

02

ÁRBOL SINTÁCTICO

La estructura jerárquica del programa. Puede ser explícito o no.

03

TABLA DE SÍMBOLOS

Lo que se sabe de cada nombre declarado: tipo, ámbito, dirección, etc.

04

TABLA DE LITERALES

Las constantes que aparecen en el código. Duh..

05

CÓDIGO INTERMEDIO

Una representación entre el fuente y el código final.
Aqui se hacen la mayoría de optimizaciones

06

ARCHIVOS TEMPORALES

Material de apoyo de vida corta..

|| [AVISO] Las implementaciones concretas varían. Esto es un conjunto de estructuras comunes. :)

TOKEN

El scanner reconoce una parte del programa fuente y la clasifica. El resultado de esa clasificación es un **token**.

Un token suele llevar dos cosas: la **categoría** a la que pertenece y un **atributo** con el texto concreto o una referencia a la tabla de símbolos.

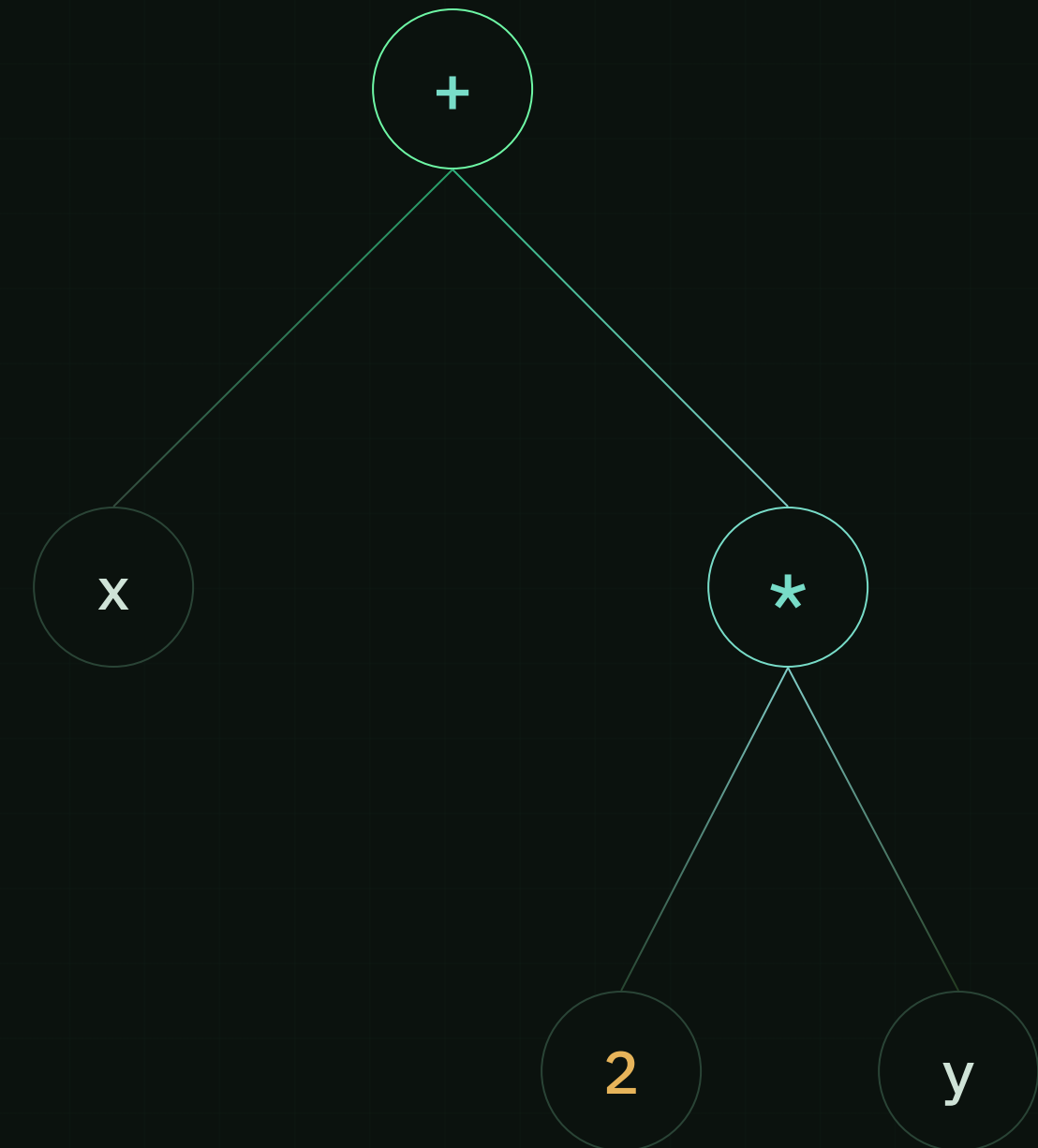


ÁRBOL SINTÁCTICO

- Normalmente vive en memoria dinámica.
- Existe una raíz desde la que se alcanza todo el programa.
- Los nodos pueden apuntar a otros nodos, como a los hijos, hermanos, o incluso a la tabla de símbolos.
- El parser puede construirlo conforme reconoce la estructura.
- El análisis semántico puede agregarle tipos, direcciones o atributos calculados. Como un arbolito de navidad. yay!!

|| No siempre tiene que construirse explícitamente. Hay compiladores que traducen sin hacer el árbol explícitamente.

EJEMPLO

 $x + 2 * y$ 

La multiplicación queda más abajo lo que significa que se evalúa primero.

TABLA DE SÍMBOLOS

Es la memoria del compilador sobre los nombres que aparecen en el programa. Cada vez que una fase necesita saber algo, consulta aquí.

¿QUÉ SE GUARDA?

variables

procedimientos

funciones

constantes

tipos

etc.

Las columnas dependen del lenguaje y del compilador. Puede haber dirección de memoria, tamaño, número de parámetros, línea de declaración, etc..

NOMBRE	TIPO	ÁMBITO
contador	int	global
Burbuja	func	global
aux	int	local

QUIÉN LA USA

SCANNER	<->	puede insertar el identificador al reconocerlo
PARSER	<->	registra declaraciones conforme reconoce la estructura
SEMÁNTICA	<->	consulta tipos y detecta usos incoherentes
GENERACIÓN	<->	necesita direcciones y tamaños para emitir código

|| [OJO] Una tabla hash es una implementación frecuente (no obligatoria).

TABLA DE LITERALES

Guarda las constantes que aparecen escritas en el programa: números, cadenas, caracteres. Su valor no cambia nunca durante la ejecución.

Una implementación puede mantener un **literal pool**: una sola copia de cada constante, reutilizada en todos los lugares donde aparece.

LITERAL POOL		3 ENTRADAS
INT	42	
FLOAT	3.14159	
STRING	"hola"	

|| [OJO] Puede ser independiente o estar integrada a otras estructuras.



CÓDIGO INTERMEDIO

Una representación del programa que ya no es el texto fuente, pero todavía no es el código final de la máquina. Sirve para razonar y optimizar con comodidad.

POSIBLES REPRESENTACIONES

estructuras	arreglos
listas	archivos
árboles / grafos	cuádruplas

EJEMPLO · CUÁDRUPLAS PARA `a = b + c * d`

(MULT, c, d, t1)

(ADD, b, t1, t2)

(STORE, t2, _, a)

LA OPTIMIZACIÓN PUEDE:

eliminar	código que nunca se ejecuta o cuyo resultado no se usa
reemplazar	una operación cara por otra equivalente más barata
simplificar	expresiones cuyo valor se conoce en tiempo de compilación
reordenar	instrucciones sin cambiar el resultado del programa

ARCHIVOS TEMPORALES

Material de apoyo con vida corta. El compilador los usa para pasar información de una fase a otra y luego los borra.

FASE 01

CREAR

Se abre un archivo auxiliar con un nombre irrelevante para el usuario.

->

FASE 02

ESCRIBIR

Se mete ahí un resultado parcial.
(tokens, código intermedio, tabla)

->

FASE 03

USAR

La siguiente fase lo lee como entrada y hace su trabajo.

->

FASE 04

DESCARTAR

Se elimina. Si todo salió bien, nadie nota que existió.

||| También pueden existir representaciones temporales que viven solo en memoria, como un buffer, una lista intermedia, un arreglo de cuádruplas que nunca toca el disco.

viven segundos... son amigos :)

NUESTRO COMPILADOR MICRO

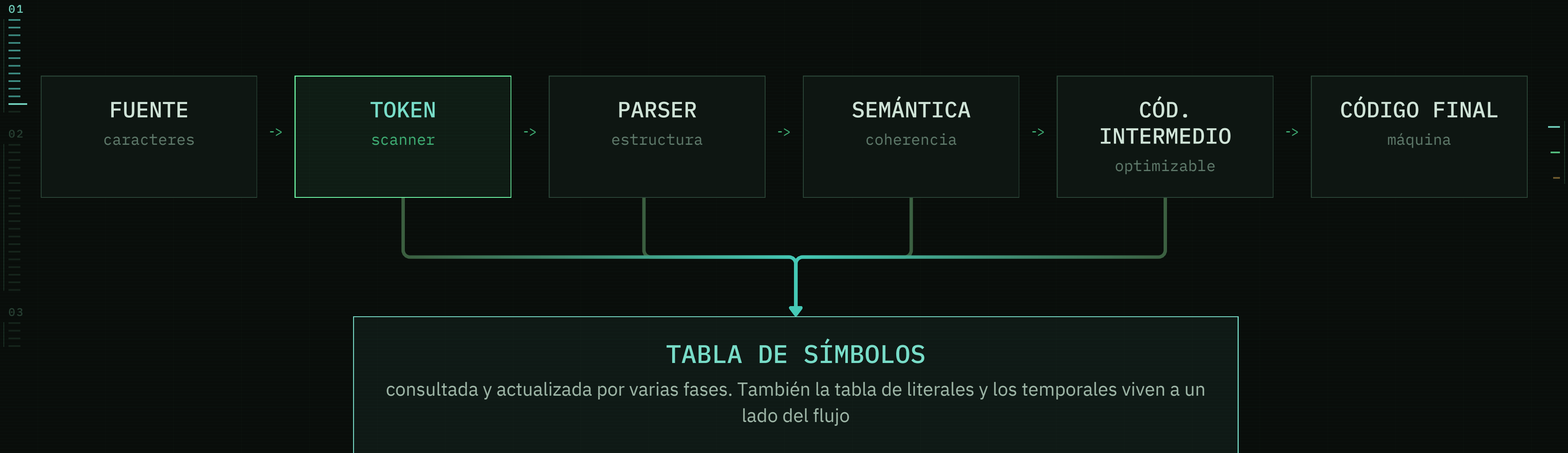
Proyecto de José Emilio Alvarado Méndez, Dayron Porras e Ian Ugalde. Sirve para ver cómo se ven estas estructuras cuando el compilador es pequeño. :)

ESTRUCTURA CONCEPTUAL	CÓMO QUEDÓ EN MICRO
Token	<code>enum token_types + token_buffer</code>
Árbol sintáctico (AST)	<code>no se construye explícitamente</code>
Tabla de símbolos	<code>arreglo + búsqueda lineal</code>
Código intermedio	<code>cuádruplas</code>
Salida intermedia	<code>archivo .s (ensamblador)</code>
Backend	<code>x86-64, ensamblado con GCC</code>

Un compilador pequeño puede simplificar muchísimo estas estructuras, lo que en un compilador industrial es una tabla hash, aquí cabe en un arreglo.

CÓMO ENCAJAN TODAS ESTAS ESTRUCTURAS

El flujo principal va de izquierda a derecha. La tabla de símbolos no pertenece a una sola fase: casi todas la consultan.



LO QUE HAY QUE RECORDAR

01 Un compilador no traduce de golpe: va guardando resultados parciales en estructuras que sobreviven entre fases.

02 El token es la unidad léxica ya clasificada, lo que el scanner le entrega al parser.

03 El árbol sintáctico captura la jerarquía del programa, pero no todo compilador lo construye explícitamente.

04 La tabla de símbolos la usan casi todas las fases. Hash es común, no obligatorio, y puede haber varias.

05 El código intermedio existe para poder optimizar cómodamente antes de bajar a la máquina.

! Ninguna de estas estructuras es obligatoria en todo compilador. Son decisiones de implementación. :]

> siguiente: ¿cómo aparecen los tokens en primer lugar?

[OK] sección 01 completa

SECCIÓN

02 ANÁLISIS LÉXICO

Antes de entender una oración primero tenemos que reconocer sus palabras.

De aquí en adelante trabajamos con la fase que convierte un chorro de caracteres en unidades con nombre.

[ILUSTRACIÓN 100% REAL]

CROP 4:3



01 · ESTRUCTURAS DE DATOS

PÁG. 03 - 15

02 · ANÁLISIS LÉXICO

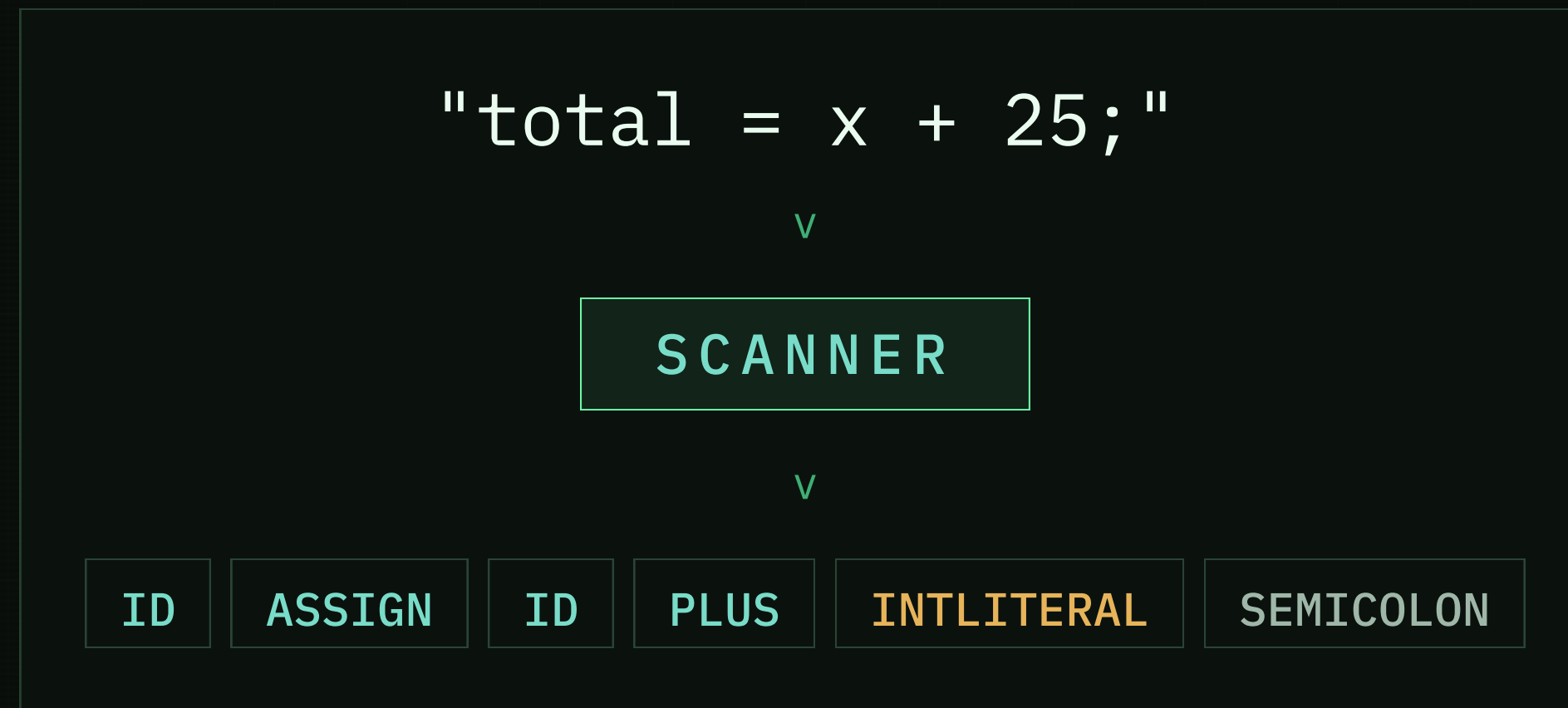
PÁG. 16 - 35

03 · HACIA LENGUAJES FORMALES

PÁG. 36 - 39

¿QUÉ HACE EL SCANNER?

El scanner recibe **caracteres**, reconoce **lexemas** y los clasifica en **tokens**.



Pensar fuente -> scanner -> tokens sirve como primera aproximación, pero la interacción real con el parser es más interesante, normalmente el scanner no produce toda la lista de una vez. La veremos en las próximas páginas.

EL CÓDIGO VISTO COMO TEXTO

burbuja.c

C · 13 LÍNEAS

```
01 void Burbuja(int N_Entradas, int vector[]){
    int indice, j, aux;
    for (indice = 0; indice < N_Entradas; indice++){
02     for (j = indice + 1; j < N_Entradas; j++){
        if (vector[j] < vector[indice]){
            aux = vector[j];
            vector[j] = vector[indice];
            vector[indice] = aux;
        }
03     }
    }
}
```

Para el scanner esto no es un algoritmo de ordenamiento: es una secuencia de caracteres. Espacios, saltos de línea, paréntesis y letras, uno detrás de otro.

La tarea de esta fase es recorrer ese texto y decidir dónde termina cada pieza y a qué categoría pertenece.

```
> leyendo burbuja.c
> el scanner no sabe qué es ordenar :)
```

[**NO TOCA**] Aquí no vamos a explicar bubble sort. Lo que importa no es qué hace el programa.

DE CARACTERES A TOKENS

Tomemos la primera línea del programa anterior. Cada pieza recibe una categoría. El color que ves en tu editor es, en el fondo, el resultado de este mismo análisis.

```
void Burbuja(int N_Entradas, int vector[]) {
```

PIEZA A PIEZA

Burbuja	-> ID
(	-> LPAREN
N_Entradas	-> ID
,	-> COMMA
int	-> palabra reservada

CÓDIGO DE COLOR DE ESTA PRESENTACIÓN

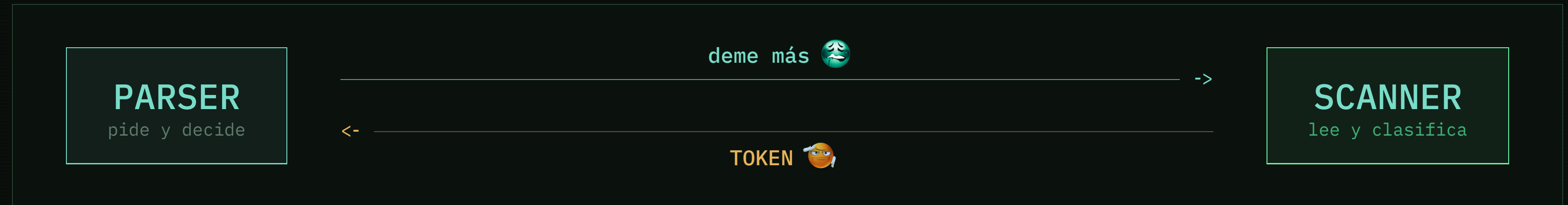
- palabra reservada
- identificador
- literal
- operador
- delimitador

Una palabra reservada como `int` se escribe igual que un identificador, pero el lenguaje se la reservó, por lo que el scanner la reconoce y le da su propia categoría.

```
// syntax highlighting = análisis léxico con colores :D
```

TRADUCCIÓN DIRIGIDA POR SINTAXIS

El scanner no tiene que producir la lista completa de tokens antes de que empiece el parser. En la práctica, es el parser quien manda: pide un token cuando lo necesita.



BAJO DEMANDA

El scanner avanza solo lo necesario. No hace falta guardar todos los tokens del archivo en memoria.

RUTINAS SEMÁNTICAS

Al reconocer una construcción, el parser puede ejecutar una rutina en el punto exacto donde tiene sentido.

SALIDA

El resultado de esas rutinas puede llegar a ser directamente código intermedio, sin árbol de por medio.

```
// traducir mientras se reconoce: de ahí el nombre :]
```

CLIENTE, PARSER Y SCANNER

Tres piezas encadenadas: quien arranca la compilación, quien reconoce la estructura y quien lee el archivo. Cada flecha es una llamada concreta.



GETTOKEN() VS. SCANNER()

La misma operación aparece con nombres distintos según el libro o el proyecto. En esta slide les voy a enseñar la implementación de mi proyecto. :)

TEORÍA · LIBRO DE TEXTO

```
Token getToken(void);
```

Devuelve un tipo Token que agrupa categoría y atributo en una sola estructura.

EN EL PARSER APARECE ASÍ

```
next_token()  
match(SEMICOLON)
```

EJEMPLO PRO · MICRO

```
token scanner(void);
```

Devuelve solo la categoría (un enum). El lexema queda aparte en un buffer global.

`next_token()` pide la siguiente unidad léxica. `match()` comprueba que el token que llegó es el que la gramática esperaba, y si no, reporta un error de sintaxis.

```
// si lee caracteres y devuelve categorías, es el scanner. da igual cómo se llame :]
```

¿QUÉ REGRESA EL SCANNER?

< tipo , atributo >

modelo abstracto con la categoría a la izquierda y el dato concreto a la derecha

<ID, "contador">

La categoría dice “es un identificador”, el atributo guarda cuál.

<INTLITERAL, "575">

El atributo puede venir como texto o ya convertido a número.

<LPAREN, "(">

Aquí el atributo sobra porque la categoría ya lo dice todo.

Los datos pueden representarse de maneras diferentes. Se pueden ver como un struct con dos campos, un entero más un buffer global, un índice a la tabla de símbolos, un par de variables, etc. :)

Lo importante no es el envase, sino que el parser reciba qué clase de cosa leyó el scanner y, cuando hace falta, cuál exactamente.

TOKEN NO ES LO MISMO QUE LEXEMA

TOKEN

ID



Una sola categoría. Es lo que el parser necesita para decidir si la construcción es válida.

LEXEMAS QUE CAEN EN ESA CATEGORÍA

x

total

N_Entradas

scanner_says_hi

Cada uno es un string que apareció en el archivo. Todos comparten el mismo token.

El **Token** es la categoría a la que pertenece la pieza (finito y conocido de antemano).

Lexema es la hilera concreta de caracteres que el scanner leyó del archivo esta vez.

¿CUÁNTOS LEXEMAS CABEN EN UN TOKEN?

Depende de la categoría. Algunas admiten exactamente una hilera, otras permiten una cantidad potencialmente no acotada de lexemas según la definición del lenguaje.

CASO A

UNO

Un solo lexema posible. La categoría y el texto son la misma cosa.

```
SEMICOLON -> ;
```

```
WHILE -> while
```

CASO B

POCOS / FINITOS

Un puñado de posibilidades listadas en la definición del lenguaje.

```
RELOP -> < <= > >= == !=
```

```
BOOL -> true false
```

CASO C

POTENCIALMENTE INFINITOS

No se pueden listar. Hay que describir la regla.

```
ID -> x, total, a1, contador_2...
```

```
INTLITERAL -> 0, 42, 575...
```

Para este tipo de categorías hay que describir patrones con expresiones regulares o autómatas, una lista de todos los identificadores posibles no existe.

GETTOKEN() EN EJECUCIÓN

El archivo fuente, visto como casillas de memoria.
El cursor avanza carácter por carácter y el scanner va acumulando hasta que puede decidir.

ENTRADA: `tabla[j] = 4 * 2;`LLAMADA: `getToken()`DEVUELVE: `ID`



LEXEMA ACUMULADO: `tabla`CURSOR EN POSICIÓN 5SP = espacio en blanco

01
Leer
tomar el carácter bajo el cursor

02
Acumular
añadirlo al buffer del lexema en curso

03
Reconocer
decidir si ya forma un lexema válido

04
Devolver
entregar el token al parser

05
Continuar
seguir desde donde quedó el cursor

> `lexema tabla` -> `token ID` · el cursor queda sobre `[`, listo para la siguiente llamada

LOS ESPACIOS NO SON TOKENS...

01 En muchos lenguajes, los espacios, tabuladores y saltos de línea no llegan al parser. El scanner los consume y sigue adelante sin producir nada.

02 No todos los lenguajes hacen esto: hay lenguajes donde la indentación sí es significativa. Pero en C, y en Micro, el blanco se descarta.

03 scanner.c

EJEMPLO OPCIONAL · MICRO

```
if (isspace(in_char)){
    continue; /* no hace nada :) */
}
```

IGNORAR ≠ SER IRRELEVANTE

Que el espacio no produzca un token no significa que no haga falta. El scanner tiene que leerlo y procesarlo para saber dónde termina el lexema anterior.

```
> leyendo ' '
> no emite token
> pero cerró el lexema anterior :]
```

...PERO SON AMIGOS :)

El espacio no se convierte en token, pero decide dónde termina una pieza y empieza la siguiente. Quítalo y el significado cambia.

• CON ESPACIO

int _ contador

int ID contador

Dos tokens. Una palabra reservada y un identificador. Es una declaración.

• SIN ESPACIO

intcontador

ID intcontador

Un solo token. Un identificador con un nombre raro. Ya no hay declaración.

El scanner puede descartar los blancos de su salida, pero solo después de procesarlos correctamente. Primero sirven de frontera, luego se tiran.

```
// trabajan gratis y se van sin despedirse  
// son amigos :)
```

FORTRAN: ¿ES UN DO?

En el FORTRAN de formato fijo los espacios en blanco no eran significativos dentro de las sentencias, y los identificadores podían escribirse con espacios adentro. El scanner no puede apoyarse en los blancos para separar piezas.

ENTRADA

DO10J=1,10,1

FORTRAN · FORMATO FIJO

FRAME 1 / 4

● AMBIGUO

D

O

1

0

J

=

1

,

1

0

,

1



CURSOR EN POSICIÓN 0 · LEXEMA ACUMULADO: (vacío)

DOS LECTURAS POSIBLES DE LA MISMA LÍNEA

DO 10 J = 1, 10, 1

un ciclo: J va de 1 a 10 de uno en uno

DO10J = 1

una asignación a una variable llamada DO10J

[?]

todavía no se puede decidir

// aquí los espacios no vienen a ayudarnos >:c

FORTRAN: ¿ES UN DO?

El scanner lee y acumula. Ya tiene DO10J, que es un identificador perfectamente válido en este lenguaje. Leer el = tampoco resuelve la construcción: una asignación también lleva igual.

ENTRADA

DO10J=1,10,1

FORTRAN · FORMATO FIJO

FRAME 2 / 4

● AMBIGUO

D

O

1

0

J

=

1

,

1

0

,

1

^

CURSOR EN POSICIÓN 5 · LEXEMA ACUMULADO: DO10J

DOS LECTURAS POSIBLES DE LA MISMA LÍNEA

DO 10 J = 1, 10, 1

sigue viva: aún no hemos visto nada que la descarte

DO10J = 1

sigue viva: DO10J es un identificador legal

[?]

todavía ambiguo >:c

> seguimos leyendo. no queda de otra.

FORTRAN: ¿ES UN DO?

El cursor llega a la primera coma. Una asignación no lleva comas ahí: la lectura DO10J = 1 queda descartada. Ese único carácter es el que decide toda la construcción.

RECONOCIDO COMO

DO 10 J = 1, 10, 1

FORTRAN · FORMATO FIJO

FRAME 3 / 4

● RESUELTO

D

0

1

0

J

=

1

,

1

0

,

1

^

CURSOR EN POSICIÓN 7 · CARÁCTER DECISIVO: ,

DOS LECTURAS POSIBLES DE LA MISMA LÍNEA

DO 10 J = 1, 10, 1

confirmada: la coma solo cabe en el encabezado del ciclo

~~DO10J = 1~~

descartada: no hay asignación con comas después

[OK]

resuelto :D

// un solo carácter, ocho posiciones más adelante, decidió la sentencia entera.

LOOKAHEAD Y RETRACT

Para resolver la ambigüedad hubo que leer más allá del lexema actual. Una vez tomada la decisión, el scanner devuelve el cursor a donde de verdad terminaba la pieza.

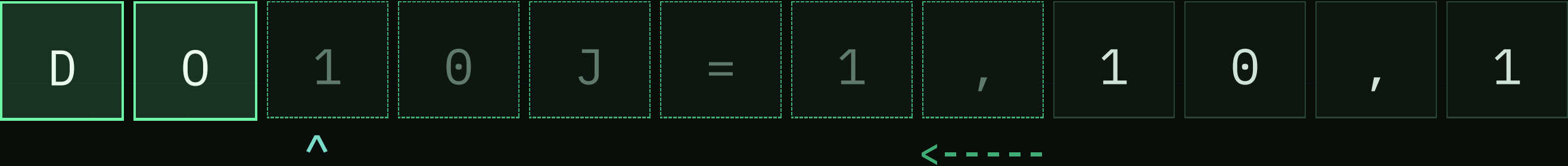
TOKEN EMITIDO

DO

FORTTRAN • FORMATO FIJO

FRAME 4 / 4

● RETRACT



RETRACT: EL CURSOR VUELVE A LA POSICIÓN 2 • lo leído de más se devuelve a la entrada

01

LEER MÁS

avanzar más allá del lexema en curso

02

OBTENER CONTEXTO

ver el carácter que desambigua

03

RECONOCER

decidir la categoría del lexema

04

RETROCEDER

devolver lo que se leyó de más

COSTO

El scanner necesita un buffer y la capacidad de retroceder. Cuánto lookahead hace falta depende del lenguaje. Casi todos usan un caracter.

// los espacios no eran tokens, pero eran buenos separadores. son amigos :)

MAXIMAL MUNCH: RECONOCER EL LEXEMA VÁLIDO MÁS LARGO

Cuando varias opciones son válidas, el scanner se queda con la más larga. Si ya leyó > y el siguiente carácter es =, emite un solo token.



// el scanner es glotón por diseño ;]



GENERADORES

Es un error común meterlas todas en el mismo saco. Dos generan scanners y dos generan parsers, por eso se usan en parejas.

GENERAN SCANNER

LEX -> scanner

FLEX -> scanner

Se les describe cada categoría con una expresión regular y generan el código que reconoce esos patrones. FLEX es la versión libre del generador.

GENERAN PARSE

YACC -> parser

BISON -> parser

Se les describe la gramática del lenguaje y generan el reconocedor de estructura. BISON es el sucesor libre de YACC.

EN PAREJA

FLEX

->

tokens

->

BISON

->

estructura + acciones

->

código intermedio

Gran parte del análisis léxico y sintáctico puede automatizarse: se describe qué se quiere reconocer y la herramienta escribe el código.

Pero también puede existir código manual: hay compiladores que escriben su scanner a mano para tener más control de errores y de rendimiento.

CONEXIÓN CON PROYECTO 1

¡REPASO!



01 El scanner lee caracteres reconociendo lexemas y los clasifica en tokens.

03 El parser normalmente pide tokens conforme los necesita, y puede traducir mientras reconoce.

05 A veces hay que leer de más para decidir, y luego retroceder: lookahead y retract.

07 LEX y FLEX generan scanners. YACC y BISON generan parsers. Suelen usarse en pareja.

02 Token es la categoría, lexema es el string concreto.

04 Los espacios suelen no producir tokens, pero delimitan lexemas. Ignorar no es ser irrelevante.

06 Ante varias opciones válidas gana la más larga: maximal munch.

! El caso FORTRAN muestra que un solo carácter, muy adelante, puede decidir toda una sentencia.

> siguiente: ¿cómo se describe formalmente lo que el scanner debe reconocer?

[OK] sección 02 completa

01
02
03

SECCIÓN

03 HACIA LENGUAJES FORMALES

Ya entendimos el scanner, falta la pregunta de cómo se describe con precisión lo que debe reconocer.

EN ESTA SECCIÓN
patrón -> regex -> autómata
símbolo vs. carácter
qué viene después

01 · ESTRUCTURAS DE DATOS
PÁG. 03 - 15

02 · ANÁLISIS LÉXICO
PÁG. 16 - 35

03 · HACIA LENGUAJES FORMALES
PÁG. 36 - 39

DE DETECCIÓN DE TOKENS A LENGUAJES FORMALES

Para reconocer algo **mecánicamente** hay que describirlo con precisión. Una categoría como ID no se puede listar porque hay que decir la regla que la genera.

01

PATRÓN

Letra seguida de letras, dígitos o guion bajo

02

-> REGEX

la misma idea sin ambigüedad:
[A-Za-z][A-Za-z0-9_]*

03

-> AUTÓMATA

una máquina que recorre la entrada y decide si encaja

04

-> SCANNER

el código que ejecuta esa máquina sobre el archivo fuente

||| LO QUE SIGUE

Expresiones regulares y autómatas se desarrollan en las siguientes clases.. o hay cosas que fueron mencionadas si llevaron Lenguajes con Chepe.. Igual aquí solo queda anotado el lugar que ocupan en la cadena.

// el scanner ya trabajaba, ahora toca describir por qué funciona :]

SÍMBOLO Y CARÁCTER NO SON SINÓNIMOS

La distinción vale la pena fijarla antes de entrar a la parte formal, porque los dos términos se usan mucho y no significan lo mismo.

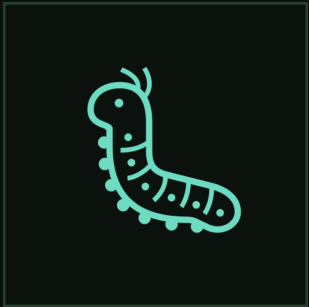
SÍMBOLOS CONVENCIONALES



advertencia



código de
producción



“feature”

Un triángulo no significa peligro por su geometría sino porque existe una convención compartida.

SÍMBOLO elemento abstracto

Elemento abstracto de un alfabeto o de un sistema formal. Su identidad no depende de cómo se dibuje.

CARÁCTER unidad de texto

Unidad de texto usada para representar información en el programa fuente. Es lo que el scanner lee del archivo.



En casos simples un carácter puede representar directamente un simbolo.

PROXIMAMENTE...

Estos apuntes terminan aquí. La siguiente formaliza el vocabulario que usamos de forma intuitiva.

||| LO QUE SIGUE FORMALIZA:

alfabetos

hileras y sus operaciones

lenguajes y operaciones sobre lenguajes

expresiones regulares

autómatas

// si esto se entendió, lo formal cuesta menos :)

> siguen los lenguajes formales



VOCABULARIO

scanner / lexer Fase que lee caracteres, reconoce lexemas, los clasifica en tokens.

lexema La hilera de caracteres que el scanner leyó del archivo.

tabla de símbolos Estructura con la información de cada nombre declarado, usado en varias fases.

código intermedio Representación entre el fuente y el código final, facilita la optimización.

lookahead Leer más allá del lexema actual para poder decidir su categoría.

maximal munch Ante varias opciones válidas gana el lexema más largo.

token Categoría léxica ya reconocida. Suele llevar un atributo asociado.

parser Fase que reconoce la estructura sintáctica a partir de los tokens.

tabla de literales Las constantes escritas en el programa.

archivo temporal Archivo auxiliar de vida corta que pasa resultados parciales entre fases.

retract Devolver a la entrada los caracteres que se leyeron de más.

símbolo Elemento abstracto de un alfabeto, no es lo mismo que el carácter que lo representa.

BIBLIOGRAFÍA

BIBLIOGRAFÍA OFICIAL DEL CURSO

01

[1]

K. Louden, *Compiler Construction: Principles and Practice*. U.S.A.: PWS Publishing Company, 1997.

02

[2]

A. Aho, M. Lam, R. Sethi and J. Ullman, *Compilers: Principles, Techniques and Tools*. Edinburgh Gate: Pearson Education Limited, 2014.

[3]

C. Fisher, R. Cytron and R. LeBlanc Jr., *Crafting a Compiler*. Addison-Wesley, Pearson, 2014.

[4]

A. Appel, *Modern Compiler Implementation in C*. Cambridge University Press, 1998.

REFERENCIAS DE APOYO

[5]

Free Software Foundation, *Flex: The Fast Lexical Analyzer*. Manual del proyecto.

[6]

Free Software Foundation, *GNU Bison Manual*.

[7]

IBM, *Open XL Fortran: Fixed Source Form*. Documentación del compilador.

[MICRO] El compilador Micro es el proyecto de José Emilio Alvarado Méndez, Dayron Porras e Ian Ugalde. Aparece de vez en cuando para ejemplificar ciertos conceptos. :)

> MUCHAS GRACIAS :]

FIN DE LOS APUNTES

Clase del 08/08/26

José Emilio Alvarado Méndez

FINAL DE LA ENTRADA

42 / 42

T H E _ E N D

^

> scan("THE END")

> accepted

> next token: EOF

bye bye! :]